Практическая работа №3. Форматы данных

Асадчий Константин Михайлович, группа 23ис-41

1. Ответы на вопросы

Текстовые форматы можно прочитать в обычном редакторе: это JSON, XML и YAML. Бинарные форматы хранят данные в виде байтов, поэтому для удобного чтения нужен декодер; в этой работе это MessagePack, CBOR и Protobuf. Компактность и скорость зависят от конкретных данных и настроек.

Схемная эволюция — изменение структуры данных так, чтобы разные версии программы могли продолжать обмениваться сообщениями. Например, новая версия Protobuf получает поле email, но читает старый файл, в котором этого поля ещё нет.

2. Таблица 1. Текстовые форматы

Формат	Размер, байт	Читаемость 1–5	Комментарии
JSON с indent=2	253	4	Нет в стандартном JSON
XML	283	3	Да: <!-- ... -->
YAML	182	5	Да: # ...

Баллы читаемости — моя оценка: YAML короче и без фигурных скобок, JSON удобен с отступами, XML перегружен тегами. В тестовом профиле комментариев нет. Для размера использован UTF-8.

Таблица 2. Все форматы по run_all.py

Формат	Размер, байт	Читаем человеком	Нужна отдельная схема
json	211	Да	Нет
xml	283	Да	Нет
yaml	182	Да	Нет
msgpack	157	Нет	Нет
cbor	159	Нет	Нет
protobuf	117	Нет	Да

Здесь JSON имеет 211 байт: в run_all.py нет indent=2. Поэтому он меньше красивого вывода step_json.py, хотя содержит те же данные. Считается только полезная нагрузка, без пяти байтов заголовка нашего протокола и сетевых заголовков.

3. Вывод отдельных тестов

Программы запущены на Python 3.12. Точные версии зависимостей записаны в файле environment.txt. Ниже показаны иллюстрации терминала по сохранённым результатам запусков. Это оформление результатов, а не фотографии или скриншоты реального окна.

step_json.py

Терминал • Иллюстрация результата

```
=== JSON ===
{
  "id": 42,
  "name": "Анна Смирнова",
  "age": 22,
  "is_student": true,
  "courses": [
    "Математика",
    "Физика",
    "Информатика"
  ],
  "address": {
    "city": "Москва",
    "zip": "101000"
  }
}
Размер: 253 байт
Десериализация успешна: Анна Смирнова
```

Иллюстрация результата, оформленная по сохранённому выводу запуска.

step_protobuf.py

Терминал • Иллюстрация результата

```
=== PROTOBUF ===
HEX: 082a1219d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0181620012a14d09cd0b0d182d0b
5d0bcd0b0d182d0b8d0bad0b02a0cd0a4d0b8d0b7d0b8d0bad0b02a16d098d0bdd184d0bed180d0bcd0b0d182d
0b8d0bad0b032160a0cd09cd0bed181d0bad0b2d0b01206313031303030
Размер: 117 байт
Десериализация успешна: Анна Смирнова
```

Иллюстрация результата, оформленная по сохранённому выводу запуска.

Для всех шести форматов проверено полное совпадение восстановленного профиля с исходным словарём. В XML типы id, age и is_student восстановлены явно; без этого XML-текст сам не превращает строки в числа и bool.

4. Передача по TCP

Сервер принимал одно сообщение на 127.0.0.1:9999 и перезапускался перед каждым клиентом. Клиент отправлял байт формата J/M/C/P, длину из четырёх байтов и тело. В полученных файлах сохранено именно тело сообщения.

Сервер: json

```
Терминал • Иллюстрация результата

Сервер слушает 127.0.0.1:9999...
=== Получено 211 байт, формат: json ===
HEX: 7b226964223a2034322c20226e616d65223a2022d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b
2d0b0222c2022616765223a2032322c202269735f73747564656e74223a20747275652c2022636f75727365732
23a205b22d09cd0b0d182d0b5d0bcd0b0d182d0b8d0bad0b0222c2022d0a4d0b8d0b7d0b8d0bad0b0222c2022d
098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad0b0225d2c20226164647265737223a207b2263697479223a2
022d09cd0bed181d0bad0b2d0b0222c20227a6970223a2022313031303030227d7d
TEXT: {"id": 42, "name": "Анна Смирнова", "age": 22, "is_student": true, "courses":
["Математика", "Физика", "Информатика"], "address": {"city": "Москва", "zip": "101000"}}
DATA: {'id': 42, 'name': 'Анна Смирнова', 'age': 22, 'is_student': True, 'courses':
['Математика', 'Физика', 'Информатика'], 'address': {'city': 'Москва', 'zip': '101000'}}
[СОХРАНЕНО] received/message_json.json (211 байт)
```

Иллюстрация результата, оформленная по сохранённому выводу запуска.

Сервер: protobuf

```
Терминал • Иллюстрация результата

Сервер слушает 127.0.0.1:9999...
=== Получено 117 байт, формат: protobuf ===
HEX: 082a1219d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0181620012a14d09cd0b0d182d0b
5d0bcd0b0d182d0b8d0bad0b02a0cd0a4d0b8d0b7d0b8d0bad0b02a16d098d0bdd184d0bed180d0bcd0b0d182d
0b8d0bad0b032160a0cd09cd0bed181d0bad0b2d0b01206313031303030
TEXT: ?*??Анна Смирнова?? ?*?Математика*
Физика*?Информатика2?

Москва??101000
DATA: {'id': 42, 'name': 'Анна Смирнова', 'age': 22, 'is_student': True, 'courses':
['Математика', 'Физика', 'Информатика'], 'address': {'city': 'Москва', 'zip': '101000'}}
[СОХРАНЕНО] received/message_protobuf.pb (117 байт)
```

Иллюстрация результата, оформленная по сохранённому выводу запуска.

В этом запуске Protobuf удалось декодировать как UTF-8, но в строке есть управляющие байты: в отчёте они показаны как \\xNN. Это не обычный читаемый текст. Поэтому ожидаемая в примере надпись «не текст» появляется не для каждого бинарного сообщения. Также успешно переданы MessagePack и CBOR. recv_exact() читает весь заголовок и тело: один вызов recv() не обязан вернуть все запрошенные байты TCP. В декодере Protobuf восстановлены все поля исходного профиля, а не только id, name и courses.

5. Полученные файлы и схемная эволюция

Файл в received/	Размер, байт	Просмотр
message_cbor.cbor	159	Бинарное сообщение
message_json.json	211	Текст JSON
message_msgpack.msgpack	157	Бинарное сообщение
message_protobuf.pb	117	Бинарное сообщение

message_json.json открывается как читаемый JSON. .pb, .msgpack и .cbor нужно читать подходящим декодером; отдельные фрагменты текста могут быть видны и внутри бинарного файла, но это не делает весь файл обычным текстом.

Порядок проверки схемной эволюции

Сначала сгенерирован profile_pb2.py по схеме без email и по TCP сохранён message_protobuf.pb. Затем в Profile добавлено string email = 7, код сгенерирован повторно, а старый .pb-файл оставлен без изменения. Новый check_evolution.py прочитал старые данные.

Терминал • Иллюстрация результата

```
Старый файл прочитан новой схемой
Имя: Анна Смирнова
Email (новое поле): ''
Проверка обратной совместимости: OK
```

Иллюстрация результата, оформленная по сохранённому выводу запуска.

Новое поле отсутствовало в старом сообщении, поэтому для string в proto3 получена пустая строка. Старые номера полей не менялись; номер удалённого поля нельзя повторно использовать для другого смысла.

Использованные файлы

В date_format/ есть data.py, profile.proto, сгенерированный profile_pb2.py, шесть step_*.py, server.py, client.py и run_all.py. Также использованы formats.py, check_evolution.py, requirements.txt, README.md, старый текст схемы, журналы запусков и received/ с данными и отчётом.

В разделах 3–5 использованы иллюстрации результатов. Настоящие скриншоты терминала и проводника в отчёт не включены.

6. Скорость и выводы

Дополнительно измерено время сериализации и десериализации: пять серий по 1000 операций, в таблице медиана времени одной операции в микросекундах. Сетевой обмен и печать не входят в измерение. Для Protobuf включено создание объекта из словаря, для XML — восстановление типов.

Формат	Запись, мкс	Чтение, мкс
json	3.521	3.349
xml	39.449	20.573
yaml	384.533	473.989
msgpack	1.118	1.636
cbor	3.223	2.497
protobuf	2.404	2.69

Результаты относятся к этому маленькому профилю и данной среде. Их нельзя переносить на все приложения: скорость зависит от библиотеки, структуры сообщения и нагрузки.

Почему Protobuf и MessagePack меньше JSON?

Protobuf получился 117 байт: вместо имён полей передаются номера и значения, а типы известны из схемы. MessagePack занимает 157 байт: имена ключей остаются, но числа, bool и структура кодируются компактными бинарными обозначениями. Для этого примера оба меньше JSON без отступов, который занимает 211 байт.

Какой формат проще читать?

Мне было проще читать YAML: поля расположены по строкам, а вложенность видна по отступам. JSON с отступами тоже удобен и хорошо подходит для обмена с веб-приложениями. Для передачи с заранее согласованной структурой я выбрал бы Protobuf, если важен небольшой размер сообщения.

Почему JSON якобы не поддерживает эволюцию схемы?

Такая формулировка слишком категорична. В JSON тоже можно добавлять поля без поломки старого клиента, если он игнорирует неизвестные поля, а новый код умеет подставлять значения для отсутствующих. Сам JSON этих правил не задаёт: их обеспечивает приложение или отдельная схема. В Protobuf совместимость поддерживается номерами полей и правилами формата; в опыте новая схема прочитала старый файл без ошибок.